

Software Performance Engineering

Recitation 1.2

Atharva Kirkole
Thursday, Sep 10, 2026



Homework 2 is released!

- You have a checkoff part and the actual homework part
 - Checkoff is due: 15th September 10 pm
 - Homework 2 is due: Monday September 21
-
- Project 1 beta: September 29, Tuesday
 - Project 1 final: October 15, Thursday
-
- Reach tier 25 for Project 1 beta to get **at least B grade!**

Logistics

- Because of typo in handout, I will be extending HW1 on request.
- Project 1 teams and team repos have been Created, only work done in there will be counted towards work contribution.
- Team contract due today @ 10PM (takes 5 mins)

Reminder for weekly report due every monday.

Teams

1. Kiwi Menzies + Braden Vibber
2. Thomas Laidlaw + Rivaan Ranawat
3. Yash Kalani + Hoang Lam Pham
4. Vu Cuong Dang + Oluwakorede Onakanmi
5. Carson Breissinger + Sri Nandan Nandamuri
6. Minh Nguyen + Joshua Thomas
7. Michael Biggar + Abdulelah Mulla
8. Anthony Betten + Eric Wheeler
9. Aaditya Moudgil + Rron Rexha
10. Nathan Czopp + Benjamin Gottschall + Vishesh Verma

what to mention in team contract

- Meeting schedule: when, where, how often, how long
- Missed-meeting policy
- Communication channel + expected response time
- Weekly hours per person
- Decision-making + a concrete tiebreaker on what you want
- How you divide, prioritize, and track work
- Handling different grade/effort goals

PROJECT 1: CONTINUE



```

29 void rotate_bit_matrix(uint8_t *img, const bits_t N) {
30     // Get the number of bytes per row in `img`
31     const uint32_t row_size = bits_to_bytes(N);
32
33     uint32_t w, h, quadrant;
34     for (h = 0; h < N / 2; h++) {
35         for (w = 0; w < N / 2; w++) {
36             uint32_t i = w, j = h;
37             uint8_t tmp_bit = get_bit(img, row_size, i, j);
38
39             // Move a bit from one quadrant to the next and do this
40             // for all 4 quadrants of the `img`
41             for (quadrant = 0; quadrant < 4; quadrant++) {
42                 uint32_t next_i = N - j - 1, next_j = i;
43                 uint8_t save_bit = tmp_bit;
44
45                 tmp_bit = get_bit(img, row_size, next_i, next_j);
46                 set_bit(img, row_size, next_i, next_j, save_bit);
47
48                 // Update the `i` and `j` with the next quadrant's values and
49                 // the `next_i` and `next_j` will get the new destination values
50                 i = next_i;
51                 j = next_j;
52             }
53         }
54     }
55
56     return;
57 }

```

First step: block-wise rotation

A	B	C	D
E	F	G	H
I	J	K	L
M	N	O	P

(a) Original image

M	B	C	A
E	F	G	H
I	J	K	L
P	N	O	D

(b) Image after translating blocks

Next step: in-block rotation

M	B	C	A
E	F	G	H
I	J	K	L
P	N	O	D

(b) Image after translating blocks

Σ	B	C	▷
E	F	G	H
I	J	K	L
⊔	N	O	⊔

(c) Image after rotating blocks

Next step: in-block rotation

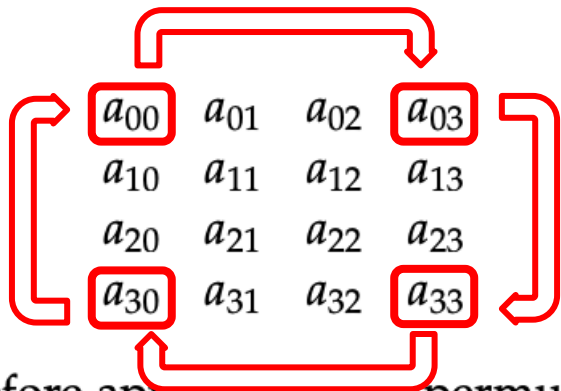
$$\begin{array}{cccc} a_{00} & a_{01} & a_{02} & a_{03} \\ a_{10} & a_{11} & a_{12} & a_{13} \\ a_{20} & a_{21} & a_{22} & a_{23} \\ a_{30} & a_{31} & a_{32} & a_{33} \end{array}$$

(a) before applying cyclic permutation

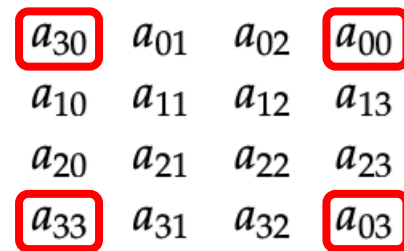
$$\begin{array}{cccc} a_{30} & a_{01} & a_{02} & a_{00} \\ a_{10} & a_{11} & a_{12} & a_{13} \\ a_{20} & a_{21} & a_{22} & a_{23} \\ a_{33} & a_{31} & a_{32} & a_{03} \end{array}$$

(b) after applying cyclic permutation

Next step: in-block rotation



(a) before applying cyclic permutation



(b) after applying cyclic permutation

One possible approach: row-column-row

- An algorithm for rotating block of bits
- Basic idea:
 - rotate row r left by $r + 1$
 - rotate column c down by $c + 1$
 - rotate row r left by r

Row-column-row algorithm

- An algorithm for rotating block of bits
- Basic idea:
 - rotate row r left by $r + 1$
 - rotate column c down by $c + 1$
 - rotate row r left by r

	0	1	2	3
0	A	B	C	D
1	E	F	G	H
2	I	J	K	L
3	M	N	O	P

Row-column-row algorithm

- An algorithm for rotating block of bits
- Basic idea:
 - ❑ **rotate row r left by $r + 1$**
 - ❑ rotate column c down by $c + 1$
 - ❑ rotate row r left by r

	0	1	2	3
0	A	B	C	D
1	E	F	G	H
2	I	J	K	L
3	M	N	O	P

Row-column-row algorithm

- An algorithm for rotating block of bits
- Basic idea:
 - ❑ **rotate row r left by $r + 1$**
 - ❑ rotate column c down by $c + 1$
 - ❑ rotate row r left by r

	0	1	2	3
0	B	C	D	A
1	E	F	G	H
2	I	J	K	L
3	M	N	O	P

Row-column-row algorithm

- An algorithm for rotating block of bits
- Basic idea:
 - ❑ **rotate row r left by $r + 1$**
 - ❑ rotate column c down by $c + 1$
 - ❑ rotate row r left by r

	0	1	2	3
0	B	C	D	A
1	E	F	G	H
2	I	J	K	L
3	M	N	O	P

Row-column-row algorithm

- An algorithm for rotating block of bits
- Basic idea:
 - ❑ **rotate row r left by $r + 1$**
 - ❑ rotate column c down by $c + 1$
 - ❑ rotate row r left by r

	0	1	2	3
0	B	C	D	A
1	G	H	E	F
2	I	J	K	L
3	M	N	O	P

Row-column-row algorithm

- An algorithm for rotating block of bits
- Basic idea:
 - ❑ **rotate row r left by $r + 1$**
 - ❑ rotate column c down by $c + 1$
 - ❑ rotate row r left by r

	0	1	2	3
0	B	C	D	A
1	G	H	E	F
2	I	J	K	L
3	M	N	O	P

Row-column-row algorithm

- An algorithm for rotating block of bits
- Basic idea:
 - ❑ **rotate row r left by $r + 1$**
 - ❑ rotate column c down by $c + 1$
 - ❑ rotate row r left by r

	0	1	2	3
0	B	C	D	A
1	G	H	E	F
2	L	I	J	K
3	M	N	O	P

Row-column-row algorithm

- An algorithm for rotating block of bits
- Basic idea:
 - ❑ **rotate row r left by $r + 1$**
 - ❑ rotate column c down by $c + 1$
 - ❑ rotate row r left by r

	0	1	2	3
0	B	C	D	A
1	G	H	E	F
2	L	I	J	K
3	M	N	O	P

Row-column-row algorithm

- An algorithm for rotating block of bits
- Basic idea:
 - ❑ **rotate row r left by $r + 1$**
 - ❑ rotate column c down by $c + 1$
 - ❑ rotate row r left by r

Rotate by 4 = no change

	0	1	2	3
0	B	C	D	A
1	G	H	E	F
2	L	I	J	K
3	M	N	O	P

Row-column-row algorithm

- An algorithm for rotating block of bits
- Basic idea:
 - rotate row r left by $r + 1$
 - **rotate column c down by $c + 1$**
 - rotate row r left by r

	0	1	2	3
0	B	C	D	A
1	G	H	E	F
2	L	I	J	K
3	M	N	O	P

Row-column-row algorithm

- An algorithm for rotating block of bits
- Basic idea:
 - rotate row r left by $r + 1$
 - **rotate column c down by $c + 1$**
 - rotate row r left by r

	0	1	2	3
0	M	C	D	A
1	B	H	E	F
2	G	I	J	K
3	L	N	O	P

Row-column-row algorithm

- An algorithm for rotating block of bits
- Basic idea:
 - rotate row r left by $r + 1$
 - **rotate column c down by $c + 1$**
 - rotate row r left by r

	0	1	2	3
0	M	C	D	A
1	B	H	E	F
2	G	I	J	K
3	L	N	O	P

Row-column-row algorithm

- An algorithm for rotating block of bits
- Basic idea:
 - rotate row r left by $r + 1$
 - **rotate column c down by $c + 1$**
 - rotate row r left by r

	0	1	2	3
0	M	I	D	A
1	B	N	E	F
2	G	C	J	K
3	L	H	O	P

Row-column-row algorithm

- An algorithm for rotating block of bits
- Basic idea:
 - rotate row r left by $r + 1$
 - **rotate column c down by $c + 1$**
 - rotate row r left by r

	0	1	2	3
0	M	I	D	A
1	B	N	E	F
2	G	C	J	K
3	L	H	O	P

Row-column-row algorithm

- An algorithm for rotating block of bits
- Basic idea:
 - ❑ rotate row r left by $r + 1$
 - ❑ **rotate column c down by $c + 1$**
 - ❑ rotate row r left by r

	0	1	2	3
0	M	I	E	A
1	B	N	J	F
2	G	C	O	K
3	L	H	D	P

Row-column-row algorithm

- An algorithm for rotating block of bits
- Basic idea:
 - rotate row r left by $r + 1$
 - **rotate column c down by $c + 1$**
 - rotate row r left by r

	0	1	2	3
0	M	I	E	A
1	B	N	J	F
2	G	C	O	K
3	L	H	D	P

Row-column-row algorithm

- An algorithm for rotating block of bits
- Basic idea:
 - rotate row r left by $r + 1$
 - rotate column c down by $c + 1$
 - **rotate row r left by r**

	0	1	2	3
0	M	I	E	A
1	B	N	J	F
2	G	C	O	K
3	L	H	D	P

Row-column-row algorithm

- An algorithm for rotating block of bits
- Basic idea:
 - rotate row r left by $r + 1$
 - rotate column c down by $c + 1$
 - **rotate row r left by r**

	0	1	2	3
0	M	I	E	A
1	B	N	J	F
2	G	C	O	K
3	L	H	D	P

Row-column-row algorithm

- An algorithm for rotating block of bits
- Basic idea:
 - rotate row r left by $r + 1$
 - rotate column c down by $c + 1$
 - **rotate row r left by r**

	0	1	2	3
0	M	I	E	A
1	N	J	F	B
2	G	C	O	K
3	L	H	D	P

Row-column-row algorithm

- An algorithm for rotating block of bits
- Basic idea:
 - rotate row r left by $r + 1$
 - rotate column c down by $c + 1$
 - **rotate row r left by r**

	0	1	2	3
0	M	I	E	A
1	N	J	F	B
2	G	C	O	K
3	L	H	D	P

Row-column-row algorithm

- An algorithm for rotating block of bits
- Basic idea:
 - rotate row r left by $r + 1$
 - rotate column c down by $c + 1$
 - **rotate row r left by r**

	0	1	2	3
0	M	I	E	A
1	N	J	F	B
2	O	K	G	C
3	L	H	D	P

Row-column-row algorithm

- An algorithm for rotating block of bits
- Basic idea:
 - rotate row r left by $r + 1$
 - rotate column c down by $c + 1$
 - **rotate row r left by r**

	0	1	2	3
0	M	I	E	A
1	N	J	F	B
2	O	K	G	C
3	L	H	D	P

Row-column-row algorithm

- An algorithm for rotating block of bits
- Basic idea:
 - rotate row r left by $r + 1$
 - rotate column c down by $c + 1$
 - **rotate row r left by r**

	0	1	2	3
0	M	I	E	A
1	N	J	F	B
2	O	K	G	C
3	P	L	H	D

	0	1	2	3
0	A	B	C	D
1				
2				
3				



	0	1	2	3
0	B	C	D	A
1				
2				
3				



	0	1	2	3
0				A
1	B			
2		C		
3			D	

	0	1	2	3
0				A
1				B
2				C
3				D

Row-column-row algorithm

- An algorithm for rotating block of bits
- Basic idea:
 - ❑ rotate row r left by $r + 1$
 - ❑ rotate column c down by $c + 1$
 - ❑ rotate row r left by r

	0	1	2	3
0	A	B	C	D
1	E	F	G	H
2	I	J	K	L
3	M	N	O	P

	0	1	2	3
0	M	I	E	A
1	N	J	F	B
2	O	K	G	C
3	P	L	H	D

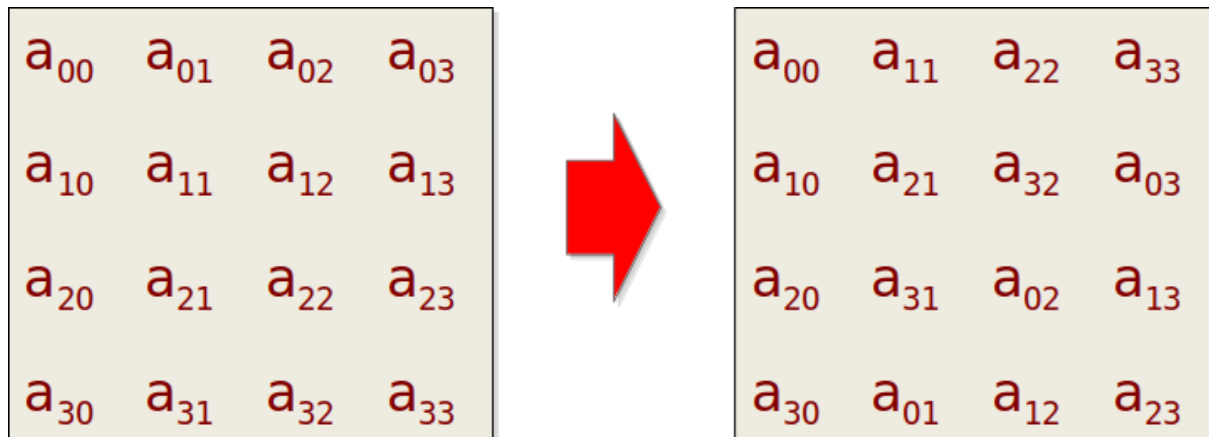
How to implement row rotation?

Operator	Description
&	AND
	OR
^	XOR (exclusive OR)
~	NOT (one's complement)
<<	shift left
>>	shift right

How to implement column rotation?

Input: $N \times N$ matrix of bits, stored in row-major order.

Goal: Circularly rotate i th column of bits up i rows.



In the example that follows, we have $N = 32$. Each row is stored in a 32-bit word, with column 0 in the most-significant bit.

Naive approach

```
const uint32_t N = 32;
const uint32_t mask = 1 << (N-1); // 1000 0000 0000 0000 0000 0000 0000 0000
uint32_t A[N];
for (int i = 0; i < N; i++){
    uint32_t col = 0;
    // gather bits in column i
    for (int j = 0; j < N; j++)
        col = col | (((A[j] << i) & mask) >> j);
    // rotate bits in column i
    col = (col << i) | (col >> (N - i));
    // put column i back
    for (int j = 0; j < N; j++)
        A[j] = (A[j] & ~(mask >> i)) |
            (((col << j) >> i) & (mask >> i));
}
```


CODING TIME!



More hints

- Loop unrolling
- Function inlining
- Algebraic identities
- Tiling
- Prefetching
-